Phase 2 Architecture

This document defines the Phase 2 architecture for BAYMAX's first local-model training experiment.

Phase 1 built the evaluation harness: 50 scripted scenarios, fake adapters, deterministic scoring, and baseline result files. Phase 2 keeps those scenarios as eval-only seed coverage, expands the benchmark to at least 200 tiered scenarios, and trains on a separate external tool-use dataset.

Goal

Phase 2 answers this question:

```
How much tool-use behavior transfers from public HuggingFace data into BAYMAX's
domain-specific task-agent contract?
```

The main comparison is:

```
base Qwen 2.5 1.5B
vs
Qwen 2.5 1.5B fine-tuned on external tool-use data + BAYMAX output adapter
```

OpenAI remains a Phase 1 reference baseline. The primary Phase 2 claim is not that the local model beats OpenAI overall. The defensible claim is:

```
Fine-tuned Qwen is characterized against base Qwen on an eval-only BAYMAX
benchmark, with tiered scores and confidence intervals.
```

Non-Goals

Phase 2 does not add:

- real Notion, Google Calendar, Gmail, or clipboard APIs
- audio input
- long-term memory
- RAG
- multi-agent orchestration
- Phase 3 scenario expansion
- LLM-as-judge scoring
- production deployment

Phase 2 is a transfer-learning experiment. It should measure what external tool-use data can and cannot teach before BAYMAX invests in a larger domain-specific training set. Underperformance against OpenAI is an expected possible outcome, not a failed phase.

Chosen Approach

Phase 2 uses:

```
external dataset + sample review + output adaptation
```

Meaning:

- Training data comes from a pre-existing HuggingFace-hosted external dataset. The current leading candidate is `Salesforce/xLAM-function-calling-60k`.
- The external dataset is kept in its native task/output format for training.
- The fine-tuned model is expected to emit the external dataset's tool-call format.
- A deterministic output adapter converts model output from the external format into BAYMAX `AgentResponse` before the eval scorer sees it.
- `scenarios/v1` and the expanded Phase 2 eval scenarios remain evaluation data only.

The output adapter is part of the Phase 2 experimental system. Marv owns the adapter implementation and translation table. Ronin consumes the adapter through its contract for eval and reporting. Adapter coverage and adapter failures must be measured separately, because adapter bugs can look like model failures.

Core Separation

The most important Phase 2 boundary is:

```
scenarios/v1 and expanded Phase 2 scenarios = evaluation only
external dataset = training source
output adapter = bridge from external output format to BAYMAX AgentResponse
```

The model must not train on:

```
scenarios/v1/*.json
scenarios/v1/scripted_responses/v1-scripted.json
results/v1-*.json
```

The scripted responses remain useful for Phase 1 scripted baseline validation, but they are not Phase 2 SFT targets.

Author-Split Discipline

Phase 2 must preserve the methodology ownership boundary:

```
Ronin owns data sourcing, SFT training, eval expansion, statistical reporting,
and benchmark analysis.

Marv owns runtime backend work, model registry interface, and output-adapter
translation-table design.
```

Ronin must not hand-author training examples or design the adapter translation table from eval-scenario knowledge. Marv must not inspect eval scenario content while doing training-data-adjacent or adapter design work.

System Flow

```
external HuggingFace tool-use dataset
        |
        v
sample review + dataset manifest
        |
        v
leakage check against all eval scenarios
        |
        v
train/validation/internal_test split
        |
        v
SFT JSONL export in external dataset format
        |
        v
Qwen 2.5 1.5B LoRA training
        |
        v
model emits external tool-call format
        |
        v
output adapter
        |
        v
BAYMAX AgentResponse
        |
        v
existing eval runner + fake adapters + scorer on eval tiers
        |
        v
base Qwen vs fine-tuned Qwen comparison with confidence intervals
```

The existing Phase 1 eval runner, fake adapters, and scorer remain the external measurement path. Phase 2 adds external dataset handling, local-model inference, output adaptation, registry metadata, leakage checks, tiered eval reporting, and comparison reporting with confidence intervals.

Contracts

Phase 2 has two output contracts.

Model Output Contract

The fine-tuned model emits the external dataset's native output format.

The exact shape depends on the chosen dataset. It may look like:

```
{
  "name": "external_calendar_create",
  "arguments": {
    "event": "linear algebra revision",
    "date": "tomorrow",
    "time": "4pm",
    "duration": "2 hours"
  }
}
```

Do not force external training examples into BAYMAX `AgentResponse` during SFT. That would turn Phase 2 into a BAYMAX-specific supervised dataset effort, which is not the chosen acquisition strategy.

BAYMAX Eval Contract

Before scoring, the output adapter must convert valid external model output into the `AgentResponse` shape already used by the eval scorer:

```
{
  "tool_calls": [
    {
      "tool": "calendar.create_event",
      "arguments": {
        "title": "linear algebra revision",
        "start_date": "2026-05-19",
        "start_time": "16:00",
        "duration_minutes": 120
      }
    }
  ],
  "message": null
}
```

Clarification and refusal outputs use no tool calls after adaptation:

```
{
  "tool_calls": [],
  "message": "Who should I send the notes to?"
}
```

The Phase 1 scorer receives only valid BAYMAX `AgentResponse` objects.

External Dataset Selection

The leading candidate is:

```
Salesforce/xLAM-function-calling-60k
```

This is a HuggingFace dataset identity: `Salesforce` is the publisher namespace, and `xLAM-function-calling-60k` is the dataset name.

The selected dataset must be recorded by exact HuggingFace identity and revision:

```
dataset_id
dataset_revision
source_split_names
license
download command
example count
tool-call format description
known schema limitations
```

Suggested manifest path:

```
data/sft/v1/dataset_manifest.json
```

If the raw downloaded dataset is large, do not commit it. Commit the manifest, hashes, split indexes, review report, leakage report, and a small sample file.

Dataset Review

Phase 2 uses sample review, not full manual curation.

Suggested report:

```
data/sft/v1/dataset_review.md
```

The review should record:

```
sample size reviewed, target 5-10% of selected examples
quality issues found
error rate, with categories: correct / wrong-tool / wrong-args / malformed
tool families covered
argument patterns covered
clarification/refusal coverage
dataset license concerns
expected transfer gaps
```

The goal is not to prove every row is perfect. The goal is to understand the dataset's quality and likely mismatch with BAYMAX before training.

Reject or revise the dataset plan if the sampled error rate is greater than 25%.

Coverage Gap Supplement

External tool-use datasets may not teach BAYMAX-specific behavior well enough. The dataset audit must explicitly check for:

```
refusal
clarification
verification
_contains-style partial matching
BAYMAX tool coverage
```

If these are missing, Phase 2 may add a small supplementary set of roughly 100-200 narrowly scoped template-generated examples. This supplement must be documented separately and must not be derived from eval scenario content.

Leakage Check

Because BAYMAX eval scenarios are the external benchmark, the data pipeline must check the external SFT dataset against all eval scenarios.

Minimum leakage checks:

```
training example IDs must not reference scenario IDs
training user text must not exactly match scenario user_input
training output must not be copied from v1 scripted responses
training metadata must not include expected_behavior or success_criteria
```

Useful later checks:

```
near-duplicate user text detection
embedding-similarity report
```

```
structural fingerprint overlap by tool/behavior/difficulty
sample-based human overlap review
```

Suggested leakage report:

```
data/sft/v1/leakage_report.json
```

The leakage report should be committed with the dataset export.

SFT Prompt And Target Format

Use chat `messages` format because Qwen 2.5 Instruct models are trained for instruction/chat-style interaction.

The prompt format should follow the chosen external dataset as much as possible. The target assistant message should be the external dataset's native tool-call output, not BAYMAX `AgentResponse`.

Example JSONL row:

```
{
  "messages": [
    {
      "role": "system",
      "content": "Return only a valid tool call in the dataset format."
    },
    {
      "role": "user",
      "content": "Schedule linear algebra revision tomorrow at 4pm for 2 hours."
    },
    {
      "role": "assistant",
      "content": "{\"name\":\"external_calendar_create\",\"arguments\":{\"event\":\"linear algebra
revision\",\"date\":\"tomorrow\",\"time\":\"4pm\",\"duration\":\"2 hours\"}}"
    }
  ],
  "metadata": {
    "external_example_id": "toolbench_000001",
    "split": "train",
    "dataset_id": "external/tool-use-dataset",
    "tool_family": "calendar"
  }
}
```

Keep model-visible training prompts faithful to the external dataset. BAYMAX runtime prompts may include BAYMAX fields such as `current_time`, `available_tools`, and `initial_state`; the output adapter must handle the format bridge during eval.

BAYMAX Eval Prompt

When evaluating against BAYMAX scenarios, the local model receives BAYMAX runtime context:

```
CURRENT_TIME:
{current_time}

AVAILABLE_TOOLS:
{available_tools_json}

INITIAL_STATE:
{initial_state_json}

USER_INPUT:
{user_input}
```

The model is still expected to emit the external dataset's output format. The adapter then maps that external output to BAYMAX `AgentResponse`.

This mismatch is intentional. Phase 2 measures transfer from public tool-use training into BAYMAX's contract through a deterministic adapter.

Training Split Strategy

The external SFT dataset uses a 70/20/10 split:

```
train: 70%
validation: 20%
internal_test: 10%
```

These splits are for training development only. They are not the final BAYMAX benchmark.

The interim external eval can use:

```
scenarios/v1
```

The Phase 2 exit eval must use an expanded benchmark with at least 200 scenarios and separate tier labels:

```
in-distribution
compositional
OOD
adversarial
```

The training split should preserve visibility across the external dataset's available metadata where possible:

```
tool family
single-tool vs multi-tool
argument complexity
clarification-like examples
refusal-like examples
```

Validation is used for training decisions such as early stopping and LoRA config selection. Internal test is used for sanity-checking the external dataset. `scenarios/v1` is useful for interim checks, but the Phase 2 headline result must be reported on the expanded tiered eval benchmark.

Dataset Manifest

Splits must be saved and reused. Do not re-randomize splits for each run.

Path:

```
data/sft/v1/dataset_manifest.json
```

Suggested shape:

```
{
  "dataset_id": "Salesforce/xLAM-function-calling-60k",
  "dataset_revision": "<huggingface_revision_or_commit>",
  "created_at": "2026-08-04T00:00:00Z",
  "source": {
    "provider": "huggingface",
    "download_command": "huggingface-cli download ...",
    "raw_cache_path": "<local_or_remote_cache_path>",
    "eval_scenario_dirs": ["scenarios/v1", "<expanded_phase2_eval_path>"],
    "eval_scripted_responses": "scenarios/v1/scripted_responses/v1-scripted.json"
  },
  "strategy": "coverage_aware_70_20_10",
  "seed": 42,
  "files": {
    "train": "data/sft/v1/train.jsonl",
    "validation": "data/sft/v1/validation.jsonl",
    "internal_test": "data/sft/v1/internal_test.jsonl",
    "review_report": "data/sft/v1/dataset_review.md",
    "leakage_report": "data/sft/v1/leakage_report.json",
    "sample": "data/sft/v1/sample.jsonl"
  },
  "counts": {
    "total": 0,
    "train": 0,
    "validation": 0,
    "internal_test": 0
```

```
  },
  "coverage": {
    "tool_family": {},
    "output_format": {},
    "single_vs_multi_tool": {},
    "clarification_like": 0,
    "refusal_like": 0
  },
  "hashes": {
    "train_sha256": "",
    "validation_sha256": "",
    "internal_test_sha256": "",
    "sample_sha256": "",
    "eval_scenarios_sha256": ""
  }
}
```

The manifest is the source of truth for the training dataset, split membership, coverage, and hashes.

Output Adapter

The output adapter translates external model output into BAYMAX `AgentResponse`.

Ownership:

```
Marv owns adapter implementation and translation-table design.
Ronin owns eval consumption, adapter failure reporting, and benchmark analysis.
```

The exact implementation path is Marv-owned and should be finalized with the backend/interface ADRs. Ronin-side code should treat the adapter as a dependency behind a stable contract, not design its mapping internals from eval scenarios.

The adapter owns:

```
external JSON parsing
external tool-name to BAYMAX tool-name mapping
external argument-name to BAYMAX argument-name mapping
unit conversions where deterministic
unsupported external tool handling
adapter failure reporting
```

The adapter must not inspect:

```
expected_behavior
success_criteria
scripted response targets
```

Those fields are eval truth. If the adapter reads them, the eval is contaminated.

Adapter Coverage Report

Before training or eval, produce an adapter coverage report:

```
data/sft/v1/adapter_coverage.json
```

It should include:

```
BAYMAX tools with direct external equivalents
BAYMAX tools with partial external equivalents
BAYMAX tools with no external equivalent
argument mappings per tool
unsupported argument fields
clarification/refusal coverage gaps
known brittle mappings
```

This report is mandatory because external datasets usually do not share BAYMAX's tool namespace or safety behavior.

Adapter Test Suite

The adapter needs unit tests before it is used in benchmark runs. Marv owns the adapter test suite. Ronin may inspect adapter outputs during eval failure analysis, but should not design adapter translation cases from specific eval scenario answers.

Tests should cover:

```
one successful mapping per BAYMAX tool
multi-tool mapping
argument renaming
unit conversion
unsupported external tool
missing required argument
clarification-like output if supported by the dataset
refusal-like output if supported by the dataset
schema-invalid adapted output
```

Adapter bugs will otherwise be indistinguishable from model failures.

Training Backend

Primary Phase 2 backend:

```
Transformers + PEFT LoRA
```

Reason:

- works naturally with cloud GPU training
- fits RunPod-style experimentation
- does not require Apple Silicon
- can evaluate base Qwen and fine-tuned Qwen through the same runner boundary

Secondary compatibility backend:

```
MLX / mlx-lm on Marv's Mac
```

MLX compatibility is useful, but it should not block the first Phase 2 training proof. The architecture should allow an MLX backend later by keeping the inference interface backend-neutral.

Training Config

Each training run must have a config that records the training recipe.

Suggested path:

```
models/qwen-1.5b-baymax-v1/training_config.json
```

Suggested shape:

```
{
  "base_model": "Qwen/Qwen2.5-1.5B-Instruct",
  "dataset": {
    "manifest": "data/sft/v1/dataset_manifest.json",
    "train_file": "data/sft/v1/train.jsonl",
    "validation_file": "data/sft/v1/validation.jsonl",
    "internal_test_file": "data/sft/v1/internal_test.jsonl",
    "external_eval_scenario_dirs": ["scenarios/v1", "<expanded_phase2_eval_path>"]
  },
  "adapter": {
    "config_file": "data/sft/v1/adapter_coverage.json",
    "version": "baymax_external_adapter_v1"
  },
  "training": {
    "epochs": 3,
    "learning_rate": 0.0002,
    "batch_size": 4,
    "max_sequence_length": 2048,
    "seed": 42
  },
```

```
  "lora": {
    "rank": 8,
    "alpha": 16,
    "dropout": 0.05,
    "target_modules": ["q_proj", "v_proj"]
  }
}
```

The most important overfitting-related knobs are epochs, learning rate, batch size, LoRA rank, and LoRA dropout.

Model Registry

The model registry records model identity and reproducibility metadata. It does not store benchmark scores.

Canonical registry file:

```
models/models.json
```

Model artifact path:

```
models/qwen-1.5b-baymax-v1/
  training_config.json
  adapter/
```

Model IDs are immutable and monotonically versioned. Never overwrite an existing model ID; a retrain creates the next version.

Suggested `models/models.json` shape:

```
{
  "models": [
    {
      "id": "qwen-1.5b-baymax-v1",
      "base": "Qwen/Qwen2.5-1.5B-Instruct",
      "adapter_path": "models/qwen-1.5b-baymax-v1/adapter",
      "adapter_format": "peft_lora",
      "trained_at": "2026-08-04T12:00:00Z",
      "training_config_file": "models/qwen-1.5b-baymax-v1/training_config.json",
      "training_config_hash": "sha256:<training_config_sha256>",
      "dataset_manifest": "data/sft/v1/dataset_manifest.json",
      "external_dataset_id": "Salesforce/xLAM-function-calling-60k",
      "external_dataset_revision": "<huggingface_revision_or_commit>",
      "output_adapter_version": "baymax_external_adapter_v1",
      "notes": "First external-data LoRA run."
    }
```

```
  ]
}
```

Large checkpoint artifacts may live outside Git if needed, but metadata should be committed so that results remain traceable.

The registry must store both `training_config_file` and `training_config_hash`. The path makes the config easy to find; the SHA-256 hash is the reproducibility invariant. If the file is edited later, the hash mismatch shows that the checkpoint no longer matches the config at that path.

Benchmark metrics live only in `results/`, never in `models/models.json`. The registry identifies models; result files measure them.

Inference Interface

Inference backends must be side-effect free. They decide what to do; they do not execute tools or mutate fake state.

Inputs:

```
current_time
user_input
available_tools
initial_state
```

Output:

```
raw model text
parsed external output if valid
adapted AgentResponse if adapter succeeds
parse_status
adapter_status
latency_ms
cost_usd or local cost estimate
```

Conceptual interface:

```
class InferenceBackend:
    def generate_response(
        self,
        *,
        current_time: str,
        user_input: str,
        available_tools: list[str],
        initial_state: dict,
    ) -> ModelGeneration:
        ...
```

`ModelGeneration` should include:

```
raw_output: str
external_output: dict | None
adapted_response: AgentResponse | None
parse_status: Literal["valid", "external_parse_failed"]
adapter_status: Literal["not_run", "adapted", "adapter_failed", "schema_invalid"]
latency_ms: int
cost_usd: float
```

The eval runner remains responsible for:

```
output adaptation
fake tool execution
final_state capture
Phase 1 scoring
Phase 2 comparison reporting
```

This keeps scripted, OpenAI, Transformers/PEFT, and MLX backends comparable.

Invalid Output Handling

Invalid external output is a model failure in primary metrics.

No JSON repair is allowed in Phase 2 primary evaluation. This measures whether the model learned the external output contract and whether the adapter can bridge that output into BAYMAX.

Failure statuses:

```
external_parse_failed
adapter_failed
schema_invalid
```

If output cannot be parsed as the external format, the runner records raw output and does not run the adapter. If adaptation fails or produces invalid `AgentResponse`, the runner records adapter failure details. The Phase 1 scorer receives only valid adapted `AgentResponse` objects.

Suggested failed-output result shape:

```
{
  "scenario_id": "calendar_create_001",
  "raw_output": "...",
  "parse_status": "valid",
  "adapter_status": "adapter_failed",
  "adapted_response": null,
  "score": null,
```

```
  "failure_reasons": ["adapter_failed"]
}
```

This prevents malformed or unadaptable output from being accidentally treated as a valid empty response.

Comparison Method

Do not change the Phase 1 scorer for Phase 2 model comparisons.

Phase 1 scorer answers:

```
Did this adapted AgentResponse pass according to the deterministic eval contract?
```

Phase 2 comparison answers:

```
How does fine-tuned Qwen + output adapter compare to base Qwen + output
adapter across the locked eval tiers?
```

The comparison layer should read result files and produce:

```
results/v2-qwen-comparison-tiered-eval.json
```

The comparison score is behavior-specific.

Tool-Call Scenarios

Applies to `tool_call` and `tool_calls` expected behavior.

```
35% decision correctness
25% tool-call accuracy
25% argument accuracy
15% safety
```

Clarification Scenarios

```
50% decision correctness
35% clarification accuracy
15% safety
```

Refusal Scenarios

```
50% decision correctness
30% refusal accuracy
20% safety
```

Decision correctness measures whether the adapted `AgentResponse` chose the right mode:

```
act
clarify
refuse
```

Safety should primarily account for hallucinated or unavailable tool calls. Premature action is already captured by decision correctness, but hallucinating unsupported tools should make a wrong decision worse.

Fine-tuned Qwen wins a scenario when its Phase 2 comparison score is higher than base Qwen's comparison score for that same scenario.

Eval Methodology Gates

Phase 2 cannot exit on a single aggregate score over the original 50 scenarios.

The benchmark used for headline Phase 2 claims must have:

```
at least 200 scenarios
tier labels for in-distribution / compositional / OOD / adversarial
separate score tables per tier
bootstrapped 95% confidence intervals for aggregate metrics
diversity metrics over tool combo / intent type / phrasing style / error mode
embedding-similarity checks against existing hand-authored scenarios
```

Before the first SFT eval run, write:

```
docs/experiments/<experiment_id>_pre_registration.md
```

That document must state what outcome counts as "the fine-tune did not work." This prevents post-hoc rationalization after results are known.

Phase 2 Success Criteria

Primary success criterion:

```
Phase 2 produces a research-defensible characterization of transfer from xLAM
style public tool-use data to BAYMAX's eval contract.
```

Exit gates:

```
eval scenario count >= 200
eval tiers reported separately: in-distribution / compositional / OOD / adversarial
aggregate metrics include bootstrapped 95% confidence intervals
pre-registered failure criterion exists before first SFT eval
```

```
adapter test suite is green
METHODOLOGY.md section 11 audit passes
```

Secondary metrics:

```
task_success_rate
average_tool_call_accuracy
average_argument_accuracy
average_clarification_accuracy
average_refusal_accuracy
average_hallucination_rate
adapter_failure_rate
invalid_external_output_rate
schema_invalid_rate
average_latency_ms
cost_usd or estimated local cost
```

The internal test split may be reported as a training sanity check. The 50-scenario `scenarios/v1` set may be used for interim checks. The headline Phase 2 benchmark must be the expanded tiered eval set.

Research Framing

The strongest Phase 2 narrative is:

```
We characterize the transfer ceiling from public tool-use datasets into a
domain-specific task-agent contract. The model trains on external tool-use data,
an output adapter bridges namespace and argument differences, and BAYMAX's
locked eval measures how much of that behavior transfers.
```

Expected weaknesses should be reported directly:

```
external dataset may lack BAYMAX tools
external dataset may lack clarification/refusal behavior
argument schemas may not match BAYMAX tools
adapter mappings may be brittle
small interim eval size limits statistical power
```

A weak fine-tuned result is still useful if the failure analysis is specific.

Expected Artifacts

Committed small artifacts:

```
data/sft/v1/dataset_manifest.json
data/sft/v1/dataset_review.md
```

```
data/sft/v1/leakage_report.json
data/sft/v1/adapter_coverage.json
data/sft/v1/sample.jsonl
docs/experiments/<experiment_id>_pre_registration.md
results/v2-qwen-base-tiered-eval.json
results/v2-qwen-sft-tiered-eval.json
results/v2-qwen-comparison-tiered-eval.json
```

Conditional artifacts:

```
data/sft/v1/train.jsonl
data/sft/v1/validation.jsonl
data/sft/v1/internal_test.jsonl
```

Commit the JSONL files only if they are repo-safe in size. If they are large, store them externally or regenerate them from the manifest and scripts.

Optional diagnostic artifacts:

```
results/v2-qwen-sft-internal-test.json
results/v2-qwen-base-v1-eval.json
results/v2-qwen-sft-v1-eval.json
results/v2-qwen-comparison-v1-eval.json
data/sft/v1/preview.md
results/v2-output-adapter-tests.json
```

Model registry artifacts:

```
models/models.json
models/qwen-1.5b-baymax-v1/training_config.json
models/qwen-1.5b-baymax-v1/adapter/
```

Large adapter/checkpoint files may be excluded from Git if needed, but the registry metadata must remain committed.

Proposed Repo Layout

Reusable tested logic:

```
src/baymax/models/
  external_dataset.py
  sft_export.py
  splits.py
  leakage.py
  registry.py
  comparison.py
```

Marv-owned runtime/adapter code:

```
<adapter path finalized by Marv's backend/interface ADR>
```

Script orchestration:

```
scripts/
  prepare_external_dataset.py
  review_external_dataset.py
  export_sft_data.py
  train_qwen_lora.py
  run_local_eval.py
  compare_model_results.py
```

Data and artifacts:

```
data/
  sft/

models/
  models.json
  qwen-1.5b-baymax-v1/

results/
```

Keep Ronin-owned importable training/eval support code in `src/baymax/models/`. Keep one-off command orchestration in `scripts/`. Keep trained artifacts outside `src/`. Keep output-adapter implementation in Marv-owned runtime code.

Benchmark Reporting

`docs/BENCHMARKING.md` should report:

```
OpenAI Phase 1 baseline on scenarios/v1
base Qwen + adapter result on expanded tiered eval
fine-tuned Qwen + adapter result on expanded tiered eval
Phase 2 comparison report
adapter failure analysis
training dataset internal test sanity check
mean +/- bootstrapped 95% CI for aggregate metrics
```

The writeup must clearly state:

```
scenarios/v1 was never used for training
training data came from an external dataset
```

```
the model emitted the external output format
the output adapter converted external output to BAYMAX AgentResponse
reported Phase 2 benchmark is external to the SFT split
results are framed as fine-tune-vs-base and transfer-learning ceiling analysis
```

Remaining Implementation Order

1. Select the external HuggingFace dataset, starting with `Salesforce/xLAM-function-calling-60k`, and record dataset ID, revision, and license.
2. Write the dataset manifest and sample-review report.
3. Build leakage checks against all eval scenarios.
4. Enumerate xLAM coverage gaps for refusal, clarification, verification, and BAYMAX-specific tool behavior.
5. Add any allowed narrow supplementary training set for documented gaps.
6. Generate the 70/20/10 training split for the external dataset.
7. Export chat-format SFT JSONL in the external dataset's native output format.
8. Coordinate with Marv on the output adapter contract; Marv owns adapter mapping implementation and tests.
9. Produce or consume the adapter coverage report.
10. Add local Transformers/PEFT inference backend.
11. Expand eval to at least 200 tiered scenarios.
12. Add diversity metrics and embedding-similarity checks for eval expansion.
13. Write the pre-registered failure criterion before first SFT eval.
14. Run base Qwen eval on the expanded tiered benchmark through the same adapter path.
15. Train first LoRA adapter.
16. Register checkpoint metadata in `models/models.json`.
17. Run fine-tuned Qwen eval on the expanded tiered benchmark.
18. Generate comparison report with bootstrapped 95% confidence intervals.
19. Run `METHODOLOGY.md` section 11 audit.
20. Update `BENCHMARKING.md`.

The first implementation pass should prioritize reproducibility, leakage prevention, adapter test coverage, and clear failure reporting over training performance.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.